# 📔 Resolución

Una vez que la nota sea entregada, encontrarás la resolución en este link de notion:

https://www.notion.so/Resoluci-n-MMIA-9ac788237c7e44c1a4cb2f5595dac10c?source=copy_link

Recuerda que existen varias formar de solucionar las actividades propuestas en los laboratorios. Toma la resolución entregada como referencia. En caso de que el link de resolución no esté público, solicita acceso en el mismo link de notion.|

# The Hello World of Neural Networks with Pytorch

The provided code demonstrates the entire process of creating a simple linear regression model, training it, making predictions, and inspecting the model's parameters. The dataset used consists of a set of numbers x and y, where x is `[-1.0, 0.0, 1.0, 2.0, 3.0, 4.0]`, and y is `[-3.0, -1.0, 1.0, 3.0, 5.0, 7.0]`. The model's objective is to approximate the linear relationship `y = 2x - 1` based on this training data.

In [28]:
```
print("x is ", [-1.0, 0.0, 1.0, 2.0, 3.0, 4.0]) #input
print("y is ", [-3.0, -1.0, 1.0, 3.0, 5.0, 7.0]) #output
```

```
x is  [-1.0, 0.0, 1.0, 2.0, 3.0, 4.0]
y is  [-3.0, -1.0, 1.0, 3.0, 5.0, 7.0]
```

Note that with a rule-based approach, we should only write a function like this

In [29]:
```
def function_with_rules(x):
    y = (2 * x) - 1
    return y

x = 4
print("For x =", x, ", y = ", function_with_rules(x=x))
```

```
For x = 4 , y =  7
```

## Importing Libraries

We import the necessary libraries, including PyTorch for building and training the neural network model and NumPy for handling numerical data.

In [30]:
```
import torch
import torch.nn as nn #neural nets
import torch.optim as optim #
import numpy as np
import os
```

# Check for GPU availability

This line of code initializes a PyTorch device based on whether CUDA, the GPU acceleration library for NVIDIA GPUs, is available on the system or not. So, the line of code essentially sets the device variable to "cuda" if CUDA is available, indicating that GPU acceleration can be used, and "cpu" otherwise, indicating that computations should be performed on the CPU. This approach allows for seamless switching between CPU and GPU computations based on availability, ensuring that the code runs on the best available hardware without errors.

In [31]:
```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu"
print("You are using", device)
```

```
You are using cpu
```

## Inspecting GPU Status in Google Colab

When you run !nvidia-smi, it displays information about the Nvidia GPU allocated to your Colab session, including details such as the GPU model, GPU memory usage, processes currently running on the GPU, and more. This command is useful for verifying that you have access to a GPU and for monitoring GPU usage during training or inference tasks in Colab.

In [32]:
```
!nvidia-smi #only works for device = "gpu"!!!!!!!
```

```
zsh:1: command not found: nvidia-smi
```

The command `nvidia-smi -L` lists the available GPUs on the system.

In [33]:
```
!nvidia-smi  -L
```

```
zsh:1: command not found: nvidia-smi
```

The command `nvidia-smi -L | wc -l` is used to count the number of GPUs available on a machine.

In [34]:
```
!nvidia-smi  -L | wc -l
```

```
zsh:1: command not found: nvidia-smi
       0
```

## Controlling GPU Allocation with

# CUDA_VISIBLE_DEVICES Configuration

This code snippet sets the environment variable CUDA_VISIBLE_DEVICES, which is used by CUDA (NVIDIA's parallel computing platform) to specify which GPUs should be made visible to CUDA-enabled applications.

Setting CUDA_VISIBLE_DEVICES is useful when you have multiple GPUs available but only want to use a subset of them for a specific task. By setting this environment variable, you can control which GPUs are utilized by CUDA-enabled applications, such as deep learning frameworks like TensorFlow or PyTorch.

In [35]:
```
num_gpus = 1 # num. gpus you want to use in this notebook

os.environ["CUDA_VISIBLE_DEVICES"] = ",".join(str(x) for x in range

print("CUDA_VISIBLE_DEVICES =", os.environ["CUDA_VISIBLE_DEVICES"])
```

```
CUDA_VISIBLE_DEVICES = 0
```

# Function to Select the Best CUDA Device

This function `get_best_cuda_device` helps in automatically selecting the most suitable CUDA-enabled GPU device for PyTorch computations. It checks for CUDA availability and, if multiple GPUs are present, it identifies the one with the most free memory to optimize performance and avoid out-of-memory errors. If no CUDA device is available, it defaults to using the CPU.

In [36]:
```
# This code is courtesy of Slin Castro (MMIA 3era Cohorte)
import torch
import pynvml

def get_best_cuda_device():
    if not torch.cuda.is_available():
        print("CUDA no está disponible. Usando CPU.")
        return torch.device("cpu")

    pynvml.nvmlInit()
    best_gpu = 0
    max_free_mem = 0

    for i in range(torch.cuda.device_count()):
        handle = pynvml.nvmlDeviceGetHandleByIndex(i)
        mem_info = pynvml.nvmlDeviceGetMemoryInfo(handle)
        free_mem = mem_info.free
        print(f"GPU {i} - Memoria libre: {free_mem / 1024**2:.2f} M

        if free_mem > max_free_mem:
            best_gpu = i
            max_free_mem = free_mem
```

```
    pynvml.nvmlShutdown()
    print(f"Seleccionando GPU {best_gpu} con más memoria libre.")
    return torch.device(f"cuda:{best_gpu}")

get_best_cuda_device()
```

```
CUDA no está disponible. Usando CPU.
```

Out[36]:
```
device(type='cpu')
```

# Model Definition

We define a simple linear regression model using PyTorch's `nn.Sequential` container. Inside the container, we have one `nn.Linear` layer, which represents a linear transformation.

In [37]:
```
# Build a simple Sequential model
model = nn.Sequential(
    nn.Linear(1, 1) # theta'*x, logits
    #no activation, identity
) #model loaded into the RAM

model.to(device)# Move the model to the GPU if available
```

Out[37]:
```
Sequential(
  (0): Linear(in_features=1, out_features=1, bias=True)
)
```

This code defines a simple neural network model using PyTorch's `nn.Sequential` container, which is a convenient way to create a sequence of neural network layers. In this case, we have only one layer:

- `nn.Linear(1, 1)` : This line defines a linear (fully connected) layer within the model. Let's break down the arguments:
  - `nn.Linear` : This is the linear layer class provided by PyTorch, which implements a linear transformation. It's essentially a matrix multiplication operation.
  - `1, 1` : The first 1 represents the number of input features, and the second 1 represents the number of output features. In other words, this layer has one input feature and produces one output feature.

So, what does this layer do? It's a linear transformation that can be expressed as $y=\theta_0+\theta_1x$ , where:

- $y$ is the output (a single number in this case).
- $\theta_1$ is the weight (a learnable parameter), and since it's 1x1, it's a scalar.

- $x$ is the input feature (a single number in this case).
- $\theta_0$ is the bias (another learnable parameter), also a scalar.

This linear layer essentially tries to learn the best values for $\theta_0$ and $\theta_1$ that allow the model to make predictions based on the input feature $x$.

In the context of this linear regression problem (predicting y based on x), this linear layer models a linear relationship between x and y. It's the core of this model, and its goal during training is to adjust the weights $\theta_0$ and $\theta_1$ to minimize the mean squared error (MSE) between the predicted values and the actual target values.

The `.to(device)` method in PyTorch is used to move tensors or models to a specific device, such as a GPU or CPU. This method is commonly used to ensure that the tensors and models are compatible with the device on which computations are performed.

When you call .to(device), you specify the device as an argument. For example, if you want to move a tensor or model to the GPU (if available), you use torch.device("cuda"). If you want to use the CPU, you specify torch.device("cpu").

# Visualizing the model

In [38]:
```
from torchsummary import summary
features = 1 # our input is a single feature
summary(model, input_size=(features,)) #output shape [-1,1], -1 is
```

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Linear-1                      [-1, 1]               2
================================================================
Total params: 2
Trainable params: 2
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.00
Forward/backward pass size (MB): 0.00
Params size (MB): 0.00
Estimated Total Size (MB): 0.00
----------------------------------------------------------------
```

The provided code snippet utilizes the summary function from the `torchsummary` library to generate a summary of the model, including information about the layers and their output shapes.

Here's a breakdown of each part of the code:

`from torchsummary import summary:` This line imports the summary function from the `torchsummary` library. This function provides a summary of a PyTorch model, including details such as the number of parameters and the output shapes of each layer.

`features = 1:` This line defines the number of features in the input data. In this case, it indicates that the input to the model consists of a single feature. This information is used to specify the input size when generating the model summary.

`summary(model, input_size=(features,)):` This line calls the summary function, passing in the model and the input size as arguments. The input_size parameter specifies the size of the input data expected by the model. In this case, it is specified as a tuple `(features,)`, indicating that the input data consists of a single feature. The summary function then analyzes the model and generates a summary, including details such as the layer types, output shapes, and the number of parameters in each layer.

The output shape [-1,1] indicates that the output of the model has a batch dimension (-1) and a single feature dimension (1). The -1 in the batch dimension represents that the batch size can vary and is determined dynamically based on the input data during inference or training.

# Loss Function and Optimizer

We define the loss function as Mean Squared Error (MSE) using `nn.MSELoss`. This is a common loss function for regression problems. We then set up the optimizer as Stochastic Gradient Descent (SGD) using `optim.SGD`. It's used to update the model's parameters during training.

In [39]:
```
# Define the loss function and optimizer
criterion = nn.MSELoss() #evaluation, loss function J
optimizer = optim.SGD(model.parameters(), lr=0.01)
```

Let's break down the code where the loss function and optimizer are defined:

- `criterion = nn.MSELoss()`: Here, we define the loss function as Mean Squared Error (MSE) using `nn.MSELoss()`. The `MSELoss` measures the mean squared difference between predicted values and actual target values. In the context of linear regression, it quantifies how well the model's predictions match the true target values. The goal during training is to minimize this loss, meaning the model aims to make its predictions as close as possible to the actual targets.
- `optimizer = optim.SGD(model.parameters(), lr=0.01)`: We

define the optimizer as Stochastic Gradient Descent (SGD) using `optim.SGD`. The parameters of this optimizer are as follows:

- `model.parameters()`: This method retrieves all the learnable parameters of the model. In the context of the linear regression model, these parameters are the weights $\theta_0$ and $\theta_1$ of the linear layer defined earlier. The optimizer will adjust these parameters during training to minimize the loss.
- `lr=0.01`: This sets the learning rate for the optimizer. The learning rate is a hyperparameter that controls the step size during the optimization process. It influences how quickly or slowly the model's parameters are updated. A smaller learning rate makes the training more stable but may require more epochs to converge, while a larger learning rate can speed up convergence but may lead to overshooting the optimal parameters.

# Data Preparation

We define our input data `xs` and output data `ys` as NumPy arrays. `xs` contains the input values, and `ys` contains the corresponding target values.

We convert these NumPy arrays into PyTorch tensors and reshape them using view to ensure they have the correct shape for PyTorch.

`.view(-1, 1)` is used to reshape the tensors. The -1 in the view method indicates that the size of that dimension is inferred from the length of the data in the tensor, and 1 specifies the new shape. In this case, it reshapes the tensors to have dimensions `(6, 1)`, where 6 represents the number of data points, and 1 represents that there is one feature for each data point

In [40]:
```
# Declare model inputs and outputs for training y = 2x - 1
xs = np.array([-1.0, 0.0, 1.0, 2.0, 3.0, 4.0], dtype=np.float32)
ys = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0], dtype=np.float32)

print(xs.shape)

# Convert NumPy arrays to PyTorch tensors and reshape xs and move t
xs = torch.tensor(xs).view(-1, 1).to(device)
ys = torch.tensor(ys).view(-1, 1).to(device)

print(xs.shape)
```

```
(6,)
torch.Size([6, 1])
```

This code prepares data for training a model using PyTorch by creating a dataset and a data loader. The dataset contains input-output pairs ( `xs` and

`ys`), and the data loader will provide batches of this data for training, with each batch containing a specified number of samples (`batch_size`) and being shuffled for each epoch (`shuffle=True`).

Let's break down what each part of the code does:

1. `from torch.utils.data import TensorDataset, DataLoader`: This line imports the `TensorDataset` class and the `DataLoader` class from the `torch.utils.data` module. These classes are commonly used in PyTorch for handling datasets and data loading during the training process.

2. `dataset = TensorDataset(xs, ys)`: This line creates a dataset object using the `TensorDataset` class. `xs` and `ys` are tensors containing input data and corresponding labels, respectively. The `TensorDataset` class allows you to combine these input-output pairs into a single dataset object.

3. `dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)`: This line creates a data loader object using the `DataLoader` class. It takes the dataset object created earlier (`dataset`) and additional parameters like `batch_size` and `shuffle`. Here, `batch_size` specifies the number of samples per batch, and `shuffle=True` indicates that the data will be shuffled randomly before being divided into batches. The data loader will then iterate over these batches during the training process.

In [41]:
```
from torch.utils.data import TensorDataset, DataLoader
# Define batch size
batch_size = 2

# Create dataset
dataset = TensorDataset(xs, ys)
print("xs: ", dataset.tensors[0], "\nys: ", dataset.tensors[1])

#reproducibility
torch.manual_seed(42)

# Create dataloader
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=Tru
```

```
xs:  tensor([[-1.],
        [ 0.],
        [ 1.],
        [ 2.],
        [ 3.],
        [ 4.]])
ys:  tensor([[-3.],
        [-1.],
        [ 1.],
        [ 3.],
        [ 5.],
        [ 7.]])
```

# Example of Automatic Differentiation in PyTorch

This code demonstrates a fundamental concept in PyTorch: automatic differentiation using the .backward() method. It shows how to define a simple mathematical function, mark a tensor for which you want to compute gradients, and then automatically calculate the derivative of the function with respect to that tensor. This is the core mechanism that enables training neural networks by calculating the gradients of the loss function with respect to the model's parameters.

In [42]:
```python
import torch

# Define a tensor x with requires_grad=True to track gradients
x = torch.tensor([7.0], requires_grad=True)

# Define the function y = x^2
y = x**2

print(x.grad) #None before backward

# Compute the gradients of y with respect to x
y.backward() # the gradient is 2*x

# Print the gradient of x
print(x.grad)
```

```
None
tensor([14.])
```

# Training

This code represents a typical training loop for a neural network model using PyTorch. Let's break it down step by step:

1. **Training Loop Initialization**: It sets the number of epochs

( `num_epochs` ) to 500, indicating how many times the entire dataset will be passed forward and backward through the neural network.

2. **Loop Over Epochs**: The outer loop iterates over each epoch from 0 to `num_epochs - 1`. During each epoch, the entire dataset is passed through the network once.

3. **Inner Loop Over Batches**: The inner loop iterates over batches of data ( `xs_batch` and `y_batch` ) obtained from the `dataloader`. `xs_batch` contains input data samples, and `y_batch` contains corresponding labels.

4. **Forward Pass**: Inside the inner loop, the input batch ( `xs_batch` ) is fed into the neural network model ( `model` ) to obtain predictions ( `outputs` ).

5. **Compute Loss**: The predicted outputs ( `outputs` ) are compared against the actual labels ( `y_batch` ) using a loss function ( `criterion` ) to calculate the loss value ( `loss` ). The loss quantifies how well the model's predictions match the true labels.

6. **Backpropagation and Parameter Update**: We clear the gradients using `optimizer.zero_grad()` to prepare for a new backward pass. This is essential to avoid the accumulation of gradients from one batch to the next, which could lead to incorrect and unstable training. It's a standard practice when training neural networks with gradient-based optimization algorithms like stochastic gradient descent (SGD). After computing the loss, the gradients of the model parameters with respect to the loss are calculated ( `loss.backward()` ), and the optimizer updates the model parameters based on these gradients ( `optimizer.step()` ). This process is called backpropagation.

In [43]:
```
# Training loop
num_epochs = 20
for epoch in range(num_epochs):

    for batch_idx, (xs_batch, y_batch) in enumerate(dataloader):

        #print("Batch", batch_idx, "xs_batch:", xs_batch, "\nys_bat
        #print("***************")
        # Forward pass
        outputs = model(xs_batch)

        # Compute the loss
        loss = criterion(outputs, y_batch) #MSE

        # Zero the gradients, perform a backward pass, and update t
        optimizer.zero_grad()
        loss.backward() #backpropagation
```

```
        optimizer.step() #Gradiend descent

    print("====================, End of Epoch", epoch)
    if epoch % 2 == 0:
        print("At Epoch ", epoch, "Loss is", loss.item()) # Use los
```

```
====================, End of Epoch 0
At Epoch  0 Loss is 0.31039080023765564
====================, End of Epoch 1
====================, End of Epoch 2
At Epoch  2 Loss is 1.386826992034912
====================, End of Epoch 3
====================, End of Epoch 4
At Epoch  4 Loss is 1.1092265844345093
====================, End of Epoch 5
====================, End of Epoch 6
At Epoch  6 Loss is 0.3399261236190796
====================, End of Epoch 7
====================, End of Epoch 8
At Epoch  8 Loss is 0.22692444920539856
====================, End of Epoch 9
====================, End of Epoch 10
At Epoch  10 Loss is 0.8594771027565002
====================, End of Epoch 11
====================, End of Epoch 12
At Epoch  12 Loss is 0.6039474606513977
====================, End of Epoch 13
====================, End of Epoch 14
At Epoch  14 Loss is 0.13777680695056915
====================, End of Epoch 15
====================, End of Epoch 16
At Epoch  16 Loss is 0.2999553978443146
====================, End of Epoch 17
====================, End of Epoch 18
At Epoch  18 Loss is 0.12496696412563324
====================, End of Epoch 19
```

# Making a Prediction:

After training, we make a prediction for the input value 10.0 by passing it through the trained model and converting the result to a Python scalar using `.item()`. The statement `with torch.no_grad()` disables gradient calculation for the operations inside the with block. Gradient calculation is used during training to update the model's parameters. However, when making predictions, we don't need to calculate gradients.

Forgetting to set the PyTorch model to evaluation mode (model.eval()) before performing inference can lead to unexpected behavior, particularly with layers like Batch Normalization or Dropout, which behave differently during training and inference. This can result in incorrect predictions or degraded model performance due to improper normalization or dropout. It's best practice to

always set the model to evaluation mode before inference to ensure consistent behavior and accurate predictions.

In [44]:
```
# Make a prediction
model.eval()
with torch.no_grad():
    predicted_value = model(torch.tensor([10.0]).to(device)) # forw

print(predicted_value)
print(predicted_value.item())  # Convert the result to a Python sca

#y = 2*x - 1 para x = 10: y:19
```

```
tensor([17.5558])
17.555845260620117
```

# Printing Model Parameters

Finally, we print the model's parameter names and their values. In this case, there's only one set of parameters corresponding to the linear transformation.

In [45]:
```
# Print model layer information
for name, param in model.named_parameters():
    print(name)
    print(param.data)# print(param.data.item())
    #print(param.grad)
print(model)

# y = 2*x -1 = theta_0 + theta_1*x
```

```
0.weight
tensor([[1.7904]])
0.bias
tensor([-0.3477])
Sequential(
  (0): Linear(in_features=1, out_features=1, bias=True)
)
```

# Actividad

For the given data, build a neural network with two hidden layers, each consisting of 'N' units, and use the sigmoid activation function. Choose the value of 'N' so that the prediction for an input of 2.5 is as close as possible to the expected value (considering the quadratic relationship between the input and output). The output layer should have a single unit with a linear activation function. Compile your model using the same optimizer and loss function as in the previous example. Train your model for 10000 epochs and calculate the mean squared error (MSE) on the training dataset.

In [49]:

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np

# 1. Reproducibilidad
torch.manual_seed(42)
np.random.seed(42)

# 2. Device: CPU o GPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu"
print("You are using:", device)

# 3. Dataset: y = x^2
xs_np = np.array([-4.0, -3.0, -2.0, -1.0, 0.0, 1.0, 2.0, 3.0, 4.0],
ys_np = np.array([16.0, 9.0, 4.0, 1.0, 0.0, 1.0, 4.0, 9.0, 16.0], d

xs = torch.tensor(xs_np).view(-1, 1).to(device)
ys = torch.tensor(ys_np).view(-1, 1).to(device)

# 4. Función para crear modelo
def create_model(N):
    model = nn.Sequential(
        nn.Linear(1, N),
        nn.Sigmoid(),
        nn.Linear(N, N),
        nn.Sigmoid(),
        nn.Linear(N, 1)
    ).to(device)
    return model

# 5. Función de entrenamiento y evaluación
def train_and_evaluate(N, num_epochs=10000, lr=0.01, verbose=False)
    torch.manual_seed(42)
    np.random.seed(42)

    model = create_model(N)
    criterion = nn.MSELoss()
    optimizer = optim.SGD(model.parameters(), lr=lr)

    for epoch in range(num_epochs + 1):
        outputs = model(xs)
        loss = criterion(outputs, ys)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if verbose and epoch % 1000 == 0:
            print(f"At Epoch {epoch}, Training Loss is {loss.item()

    model.eval()

    with torch.no_grad():
        train_predictions = model(xs)
        final_mse = criterion(train_predictions, ys)
```

```
        x_test = torch.tensor([[2.5]], dtype=torch.float32).to(devi
        predicted_value = model(x_test).item()

    expected_value = 2.5 ** 2
    abs_error = abs(expected_value - predicted_value)

    return model, train_predictions, final_mse.item(), predicted_va

# 6. Entrenamiento inicial con N = 16
N = 16
model, train_predictions, final_mse, prediction, absolute_error = t
    N=N,
    num_epochs=10000,
    lr=0.01,
    verbose=True
)

print("\nModel with N = 16:")
print(model)

print("\nFinal MSE on training dataset:", final_mse)
print("\nPrediction for x = 2.5:", prediction)
print("Expected value for x = 2.5:", 2.5 ** 2)
print("Absolute error:", absolute_error)

print("\nComparison on training data:")
print("x\tReal y\tPredicted y\tAbsolute Error")

with torch.no_grad():
    for x_real, y_real, y_pred in zip(xs, ys, train_predictions):
        error = abs(y_real.item() - y_pred.item())
        print(f"{x_real.item():.1f}\t{y_real.item():.4f}\t{y_pred.i

# 7. Comparación con varios valores de N
candidate_N = [4, 8, 16, 32, 64]

best_N = None
best_prediction = None
best_mse = None
best_abs_error = float("inf")
best_model = None
best_train_predictions = None

print("\nTesting different values of N:")
print("-" * 40)

for N in candidate_N:
    model, train_predictions, final_mse, predicted_value, abs_error
        N=N,
        num_epochs=10000,
        lr=0.01,
        verbose=False
    )

    print(f"N={N}")
```

```
    print(f"Final MSE: {final_mse:.8f}")
    print(f"Prediction x=2.5: {predicted_value:.8f}")
    print(f"Absolute error: {abs_error:.8f}")
    print("-" * 40)

    if abs_error < best_abs_error:
        best_N = N
        best_prediction = predicted_value
        best_mse = final_mse
        best_abs_error = abs_error
        best_model = model
        best_train_predictions = train_predictions

# 8. Mejor modelo
expected_value = 2.5 ** 2

print("\nBest model:")
print("Best N:", best_N)
print("Best MSE:", best_mse)
print("Best prediction for x=2.5:", best_prediction)
print("Expected value:", expected_value)
print("Best absolute error:", best_abs_error)

# 9. Tabla final del mejor modelo
print("\nComparison on training data using best model:")
print("x\tReal y\tPredicted y\tAbsolute Error")

with torch.no_grad():
    for x_real, y_real, y_pred in zip(xs, ys, best_train_prediction
        error = abs(y_real.item() - y_pred.item())
        print(f"{x_real.item():.1f}\t{y_real.item():.4f}\t{y_pred.i
```

```
You are using: cpu
At Epoch 0, Training Loss is 76.07915497
At Epoch 1000, Training Loss is 0.13353181
At Epoch 2000, Training Loss is 0.07676280
At Epoch 3000, Training Loss is 0.05037953
At Epoch 4000, Training Loss is 0.03046533
At Epoch 5000, Training Loss is 0.01712640
At Epoch 6000, Training Loss is 0.00882577
At Epoch 7000, Training Loss is 0.00440821
At Epoch 8000, Training Loss is 0.00237464
At Epoch 9000, Training Loss is 0.00148343
At Epoch 10000, Training Loss is 0.00107764

Model with N = 16:
Sequential(
  (0): Linear(in_features=1, out_features=16, bias=True)
  (1): Sigmoid()
  (2): Linear(in_features=16, out_features=16, bias=True)
  (3): Sigmoid()
  (4): Linear(in_features=16, out_features=1, bias=True)
)

Final MSE on training dataset: 0.001077334862202406
```

```
Prediction for x = 2.5: 6.137537002563477
Expected value for x = 2.5: 6.25
Absolute error: 0.11246299743652344

Comparison on training data:
x       Real y  Predicted y     Absolute Error
-4.0    16.0000 15.9859         0.014134
-3.0    9.0000  9.0307          0.030713
-2.0    4.0000  3.9705          0.029515
-1.0    1.0000  1.0129          0.012904
0.0     0.0000  -0.0177         0.017662
1.0     1.0000  1.0550          0.054996
2.0     4.0000  3.9451          0.054946
3.0     9.0000  9.0316          0.031650
4.0     16.0000 15.9874         0.012570

Testing different values of N:
----------------------------------------
N=4
Final MSE: 0.11343363
Prediction x=2.5: 6.35511589
Absolute error: 0.10511589
----------------------------------------
N=8
Final MSE: 0.00177931
Prediction x=2.5: 5.99319172
Absolute error: 0.25680828
----------------------------------------
N=16
Final MSE: 0.00107733
Prediction x=2.5: 6.13753700
Absolute error: 0.11246300
----------------------------------------
N=32
Final MSE: 0.00902530
Prediction x=2.5: 6.19059849
Absolute error: 0.05940151
----------------------------------------
N=64
Final MSE: 0.00801418
Prediction x=2.5: 6.19637585
Absolute error: 0.05362415
----------------------------------------

Best model:
Best N: 64
Best MSE: 0.00801418349146843
Best prediction for x=2.5: 6.196375846862793
Expected value: 6.25
Best absolute error: 0.05362415313720703

Comparison on training data using best model:
x       Real y  Predicted y     Absolute Error
-4.0    16.0000 15.9643         0.035720
-3.0    9.0000  9.1070          0.107047
-2.0    4.0000  3.8719          0.128129
```

```
-1.0    1.0000  0.9980          0.002013
0.0     0.0000  0.1207          0.120669
1.0     1.0000  0.9975          0.002451
2.0     4.0000  3.8725          0.127535
3.0     9.0000  9.1048          0.104810
4.0     16.0000 15.9660         0.033975
```

En esta actividad se implementó una red neuronal en PyTorch para aproximar la relación cuadrática y = x^2, usando un dataset con valores de entrada entre -4 y 4 y sus respectivas salidas cuadráticas. La arquitectura utilizada tiene dos capas ocultas con activación Sigmoid y una capa de salida lineal. Esta estructura es adecuada porque la relación entre x e y es no lineal, por lo que se requieren activaciones no lineales para aproximar la forma de la función cuadrática. El modelo se entrenó hasta la época 10000 usando `MSELoss` como función de pérdida y `SGD` como optimizador. Durante el entrenamiento, el loss disminuyó progresivamente, lo que evidencia que la red fue ajustando sus parámetros para reducir el error entre las predicciones y los valores reales. Se probaron diferentes valores de N para seleccionar la arquitectura que aproxima mejor el valor esperado para x = 2.5. Aunque el enunciado permite escoger N, se evaluaron varias opciones para justificar la selección final con base en el menor error absoluto y el MSE del conjunto de entrenamiento. Los valores evaluados fueron N = 4, 8, 16, 32 y 64. El modelo con N = 64 obtuvo la predicción más cercana para x = 2.5, con un valor aproximado de 6.1964, frente al valor esperado de 6.25. Esto representa un error absoluto aproximado de 0.0536. El MSE final en el conjunto de entrenamiento fue aproximadamente 0.0080.

La actividad muestra que una red neuronal con capas ocultas y activaciones no lineales puede aproximar una relación cuadrática. Una sola capa lineal no sería suficiente para representar correctamente y = x^2, ya que únicamente podría aprender relaciones de la forma y = wx + b. La comparación de distintos valores de N permitió observar cómo el número de neuronas ocultas influye en el desempeño del modelo. En este caso, N = 64 produjo la predicción más cercana al valor esperado para x = 2.5, manteniendo un MSE bajo en el conjunto de entrenamiento.

El modelo desarrollado cumple con los requisitos de la actividad: dos capas ocultas con N unidades, activación Sigmoid, salida lineal, `MSELoss`, `SGD` y entrenamiento hasta la época 10000. El mejor resultado para la predicción solicitada se obtuvo con N = 64, alcanzando una predicción aproximada de 6.1964 para x = 2.5, cuyo valor esperado era 6.25. Por tanto, el modelo logró aproximar de manera adecuada la relación cuadrática y = x^2.

# References